Proyecto Fenix

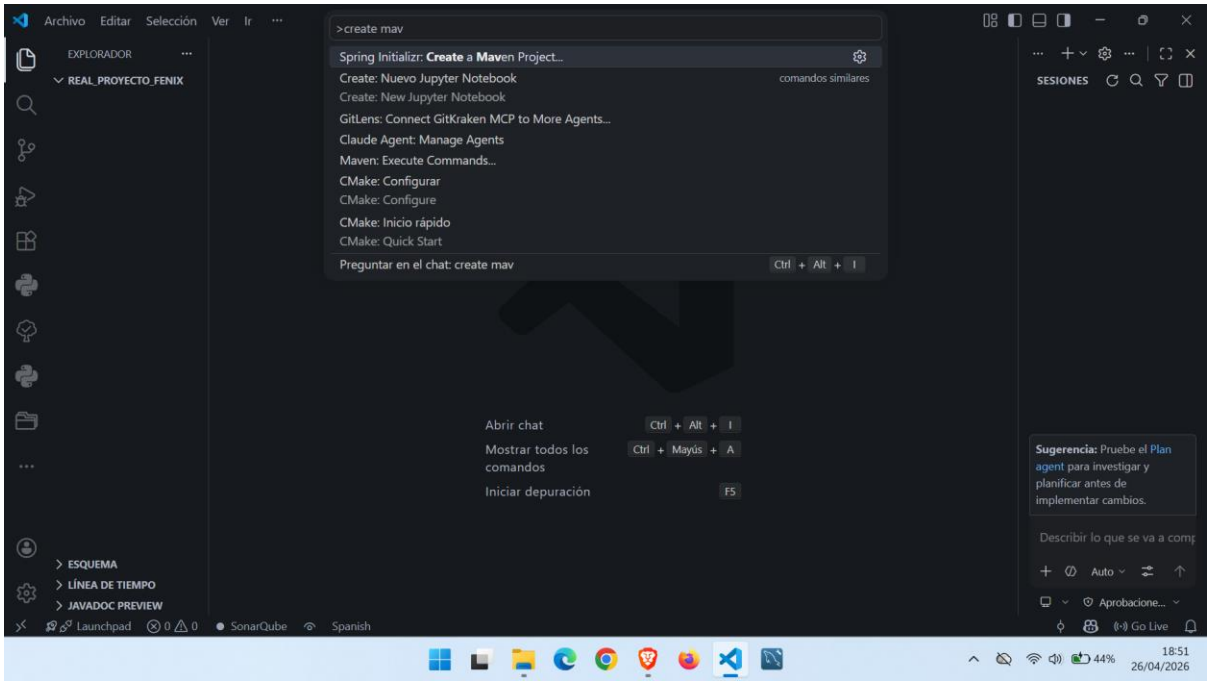
Por: Valentin Fuentes/DAWº1

Real_Proyecto_Fenix	26/04/2026 18:50	Carpeta de archivos

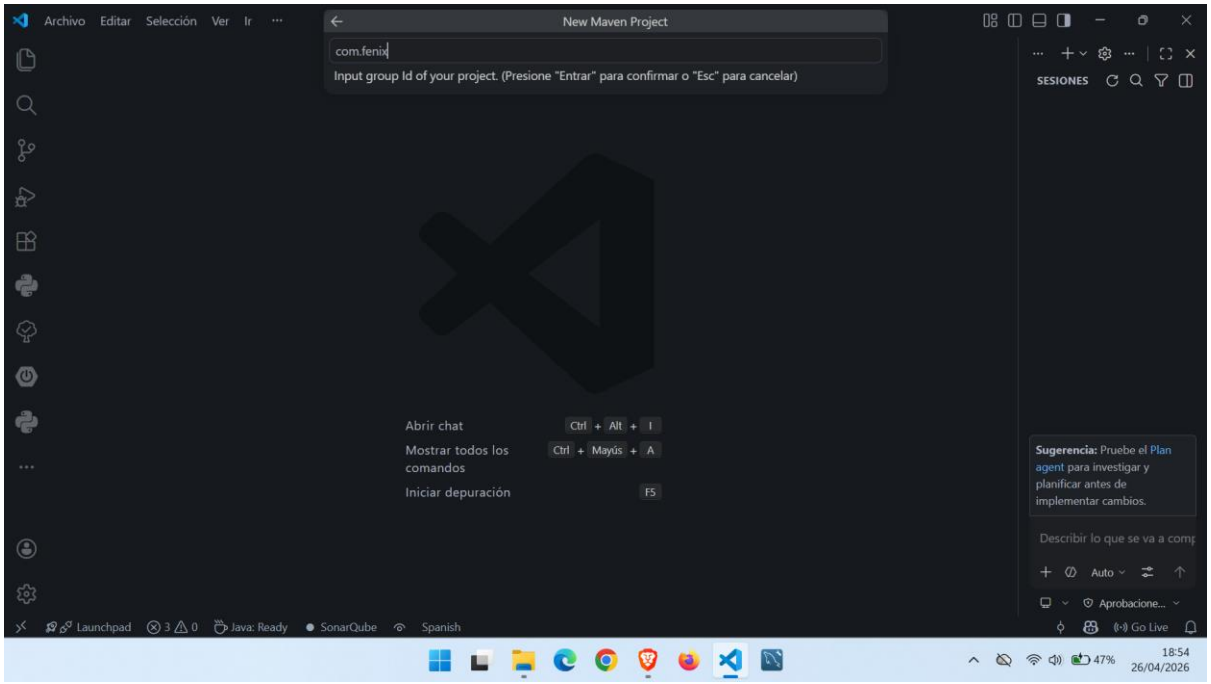
. Primero; creamos una carpeta donde estara el proyecto, puede ser en cualquier lugar del ordenador (en mi caso, la he hecho en Documentos).

. Segundo; Una vez hecho correctamente lo primero, nos deberia salir una carpeta a la izquierda, ahora usamos el comando; CRTL+Shift+P y escribimos; Create, esto nos dejara la opción; Create a Maven Project seleccionamos la version mas reciente y seguimos.

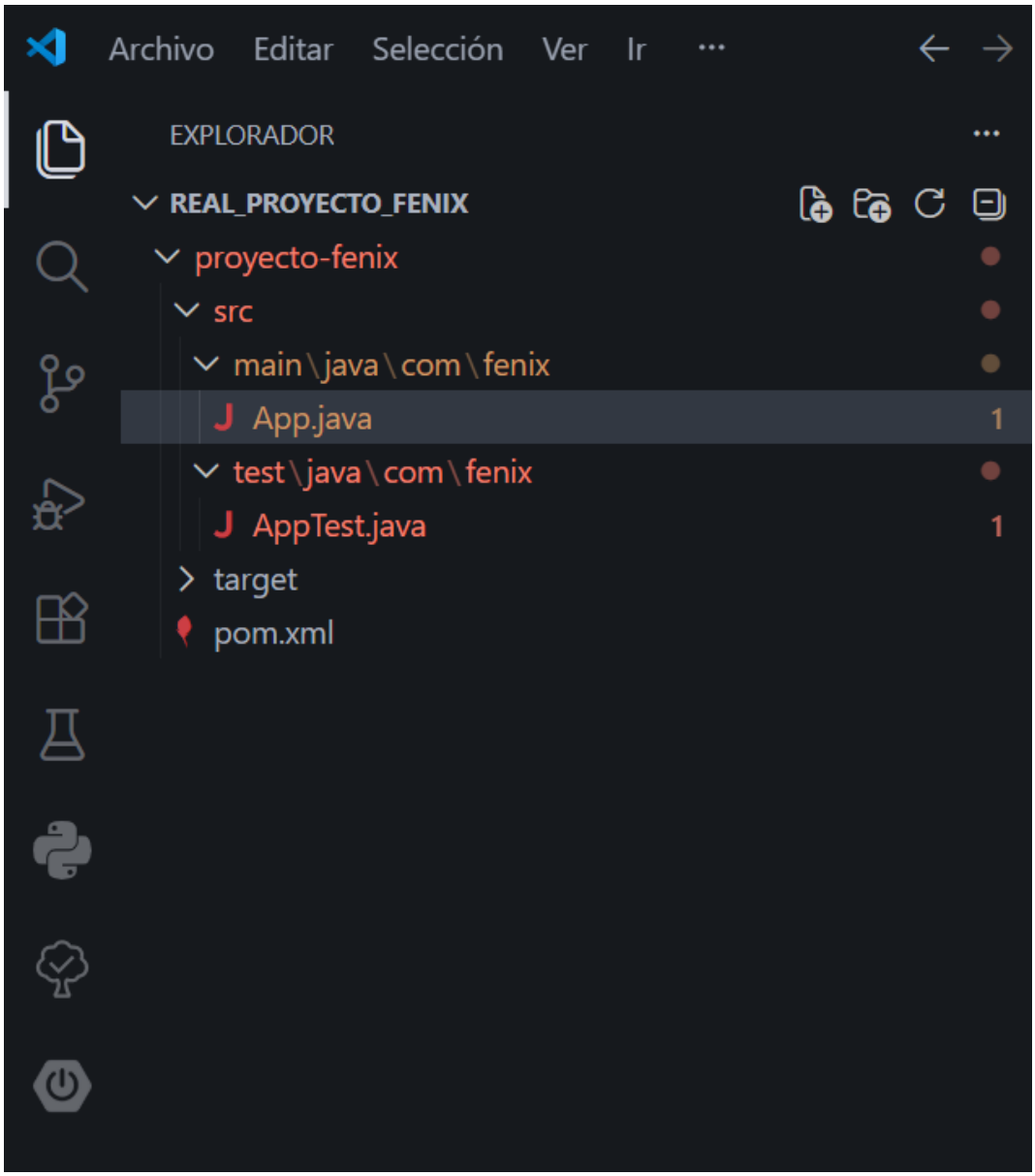
(NOTA: al hacer el proyecto Maven de esta forma, este nos generara un pom.xml de forma automatica, el cual deberemos editar para que todo funcione)



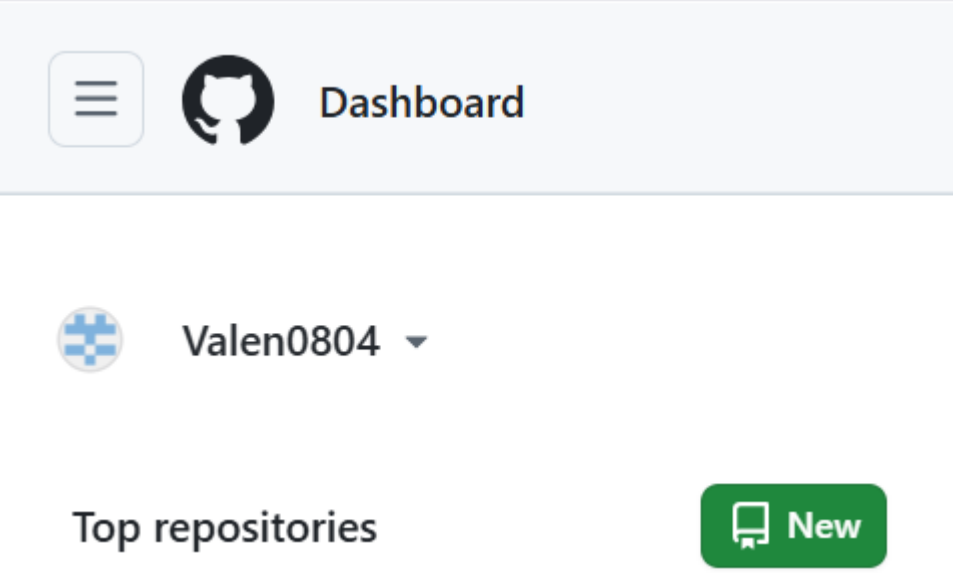
. Tercero; Despues de escribir; Create Maven Project, seleccionamos el Input group y luego el artifact ID.



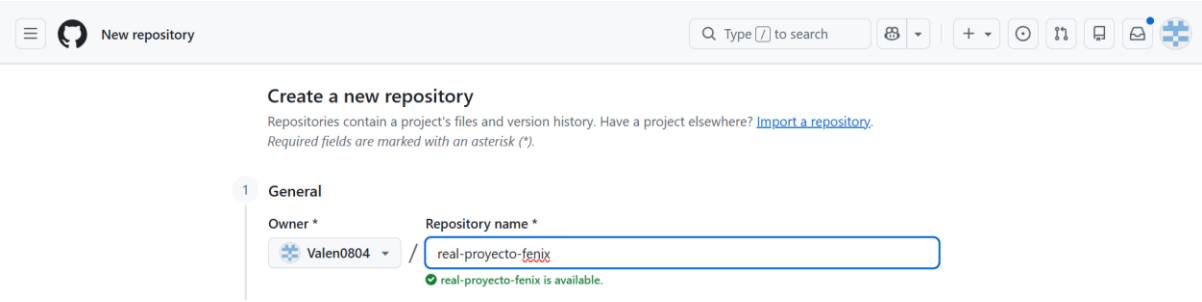
Cuarto; Despues de haber hecho todo de forma correcta nos debera salir estos archivos java, los remplazáremos por los archivos ProcesadorUsuarios.Java y EjemploPrueba.java

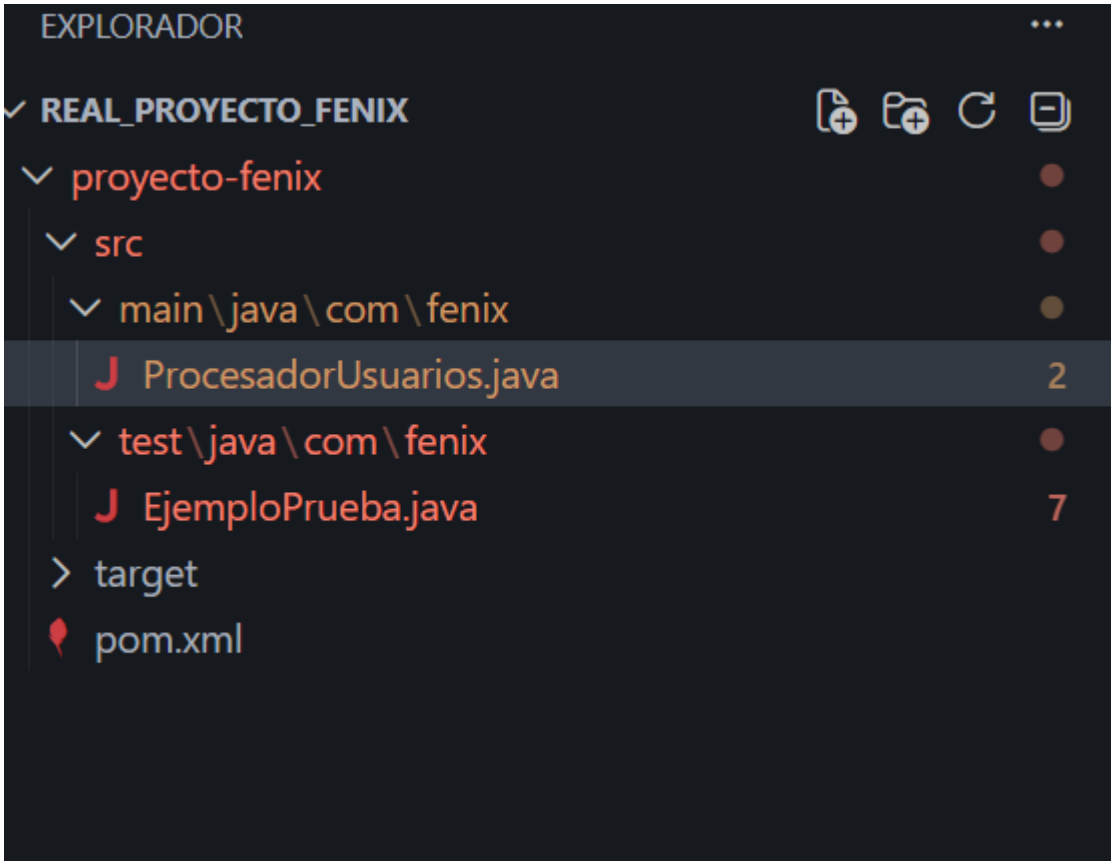


Parte de GitHub; Ahora entramos a GitHub (https://github.com) y le damos al botón verde “New” al lado de “Top repositories”



Aquí le colocamos un nombre a nuestro proyecto, lo dejamos sin "README" y sera un repositorio público aqui solo bajamos y damos click en el botón verde "create repository".





Una vez tengamos el repositorio creado, vamos a comenzar a "Reparar" estos archivos para que puedan compilar y funcionar correctamente.

```
PROBLEMAS 14   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   ···   JavaSE-25 LTS   +   ···

Página de códigos activa: 65001

C:\Users\alumno\Documents\Real_Proyecto_Fenix>git init
```

Despues de haber colocado el código “bueno” iniciamos el repositorio con este comando.

Usamos el git add . Para preparar los cambios para hacer un commit.

```
C:\Users\alumno\Documents\Real_Proyecto_Fenix>git add .
```

Y ahora con este comando hacemos el primer commit

```
git commit -m "Commit inicial: Código Heredado"
```

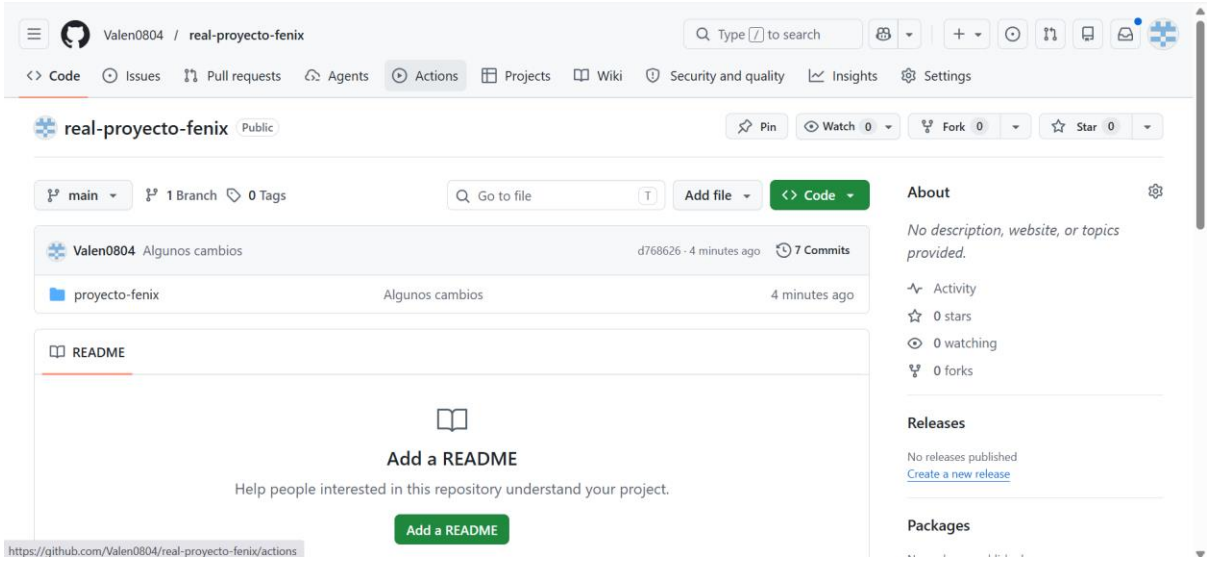
Y con este comando conectamos el proyecto a GitHub

```
C:\Users\alumno\Documents\Real_Proyecto_Fenix>git remote add origin https://github.com/Valen0804/real-pro
yecto-fenix.git
```

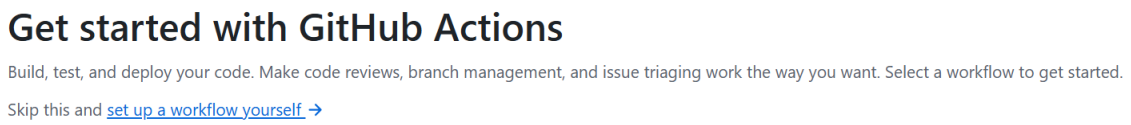
(El “BUILD SUCCESS” significa que todo esta bien.)

```
------------------------
BUILD SUCCESS
------------------------
```

Aqui nos dirigiremos a la seccion que se llama “Actions” y daremos click alli.

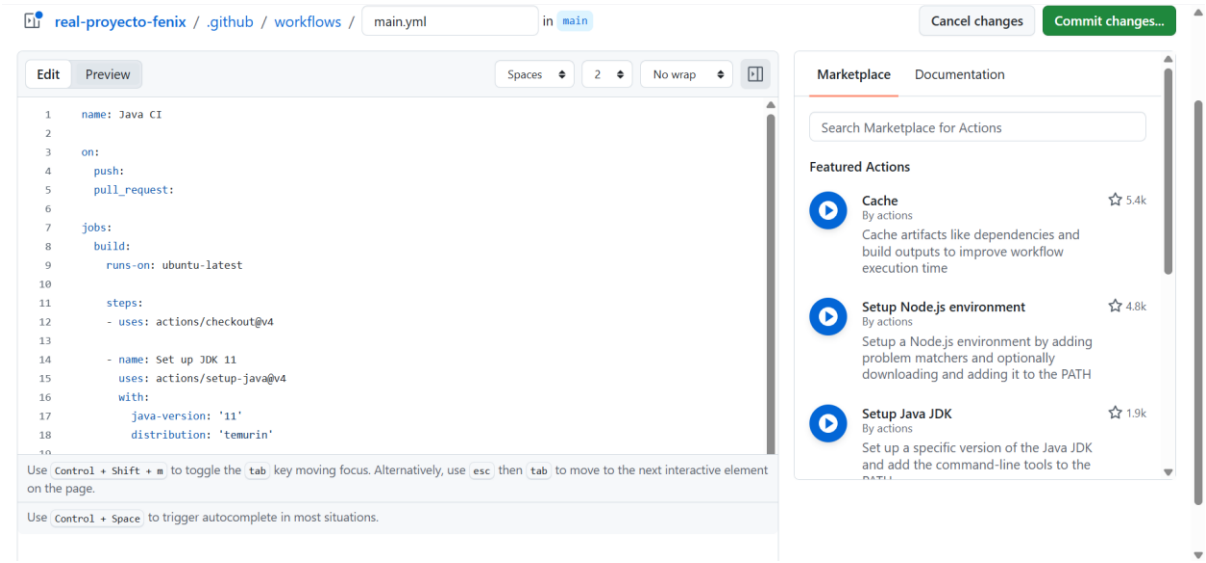


Aquí le damos click a la opción “set up a workflow yourself”

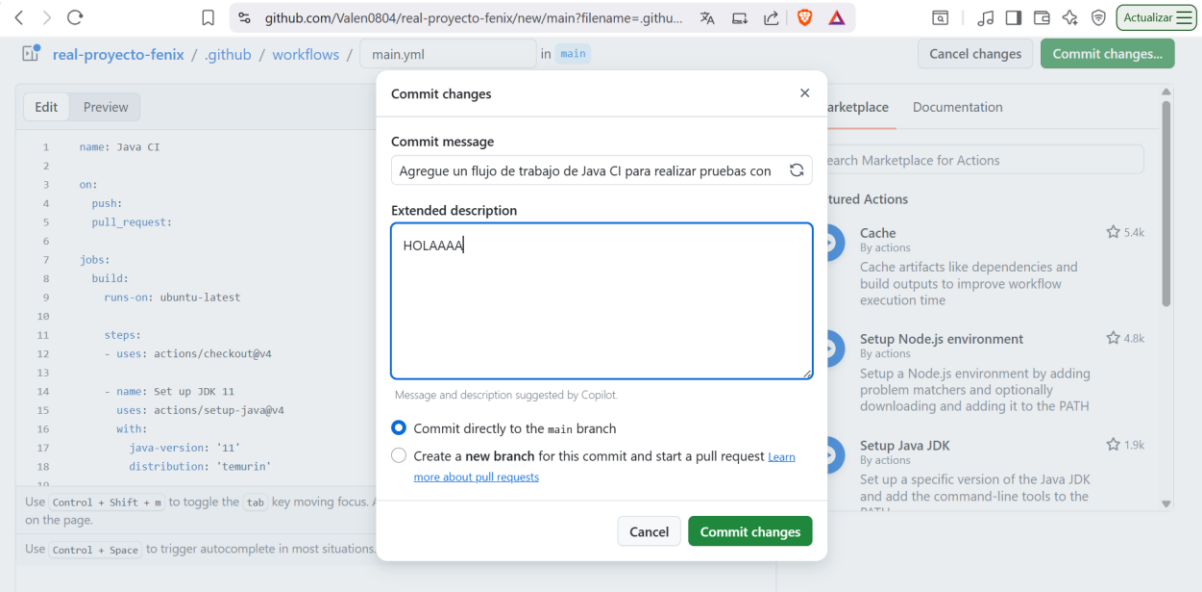


Despues de hacer click, nos dirigira a esta pestaña, donde hay un main.yml generado por nosotros el cual llenaremos para que pueda

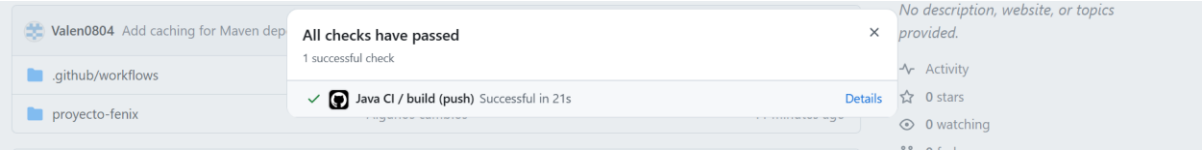
hacer cosas como; descargar el código, instalar java y poder hacer un mvn test, le damos al botón “Commit changes” y seguimos.



Despues de haberle dado click, nos llevara a esta ventana, nos generara un mensaje automaticamente, tambien le podras poner una descripción mas extendida, damos click en “Commit changes” y nos guardara lo que hemos hecho.



Si despues de haber hecho todo y nos sale este mensaje, eso quiere decir que todo esta correcto, ahora nuestro proyecto puede; Ejecutar y validar lógica automáticamente, Tener pruebas automáticas (JUnit), Integración continua (GitHub Actions), Proyecto mantenible (refactorizado), Documentación automática (Javadoc), Control de versiones con Git.



Ahora generamos el javadoc, usando este comando, asi podremos colocar nuestra autoria acerca del proyecto, o dejar alguna explicación de como funciona.

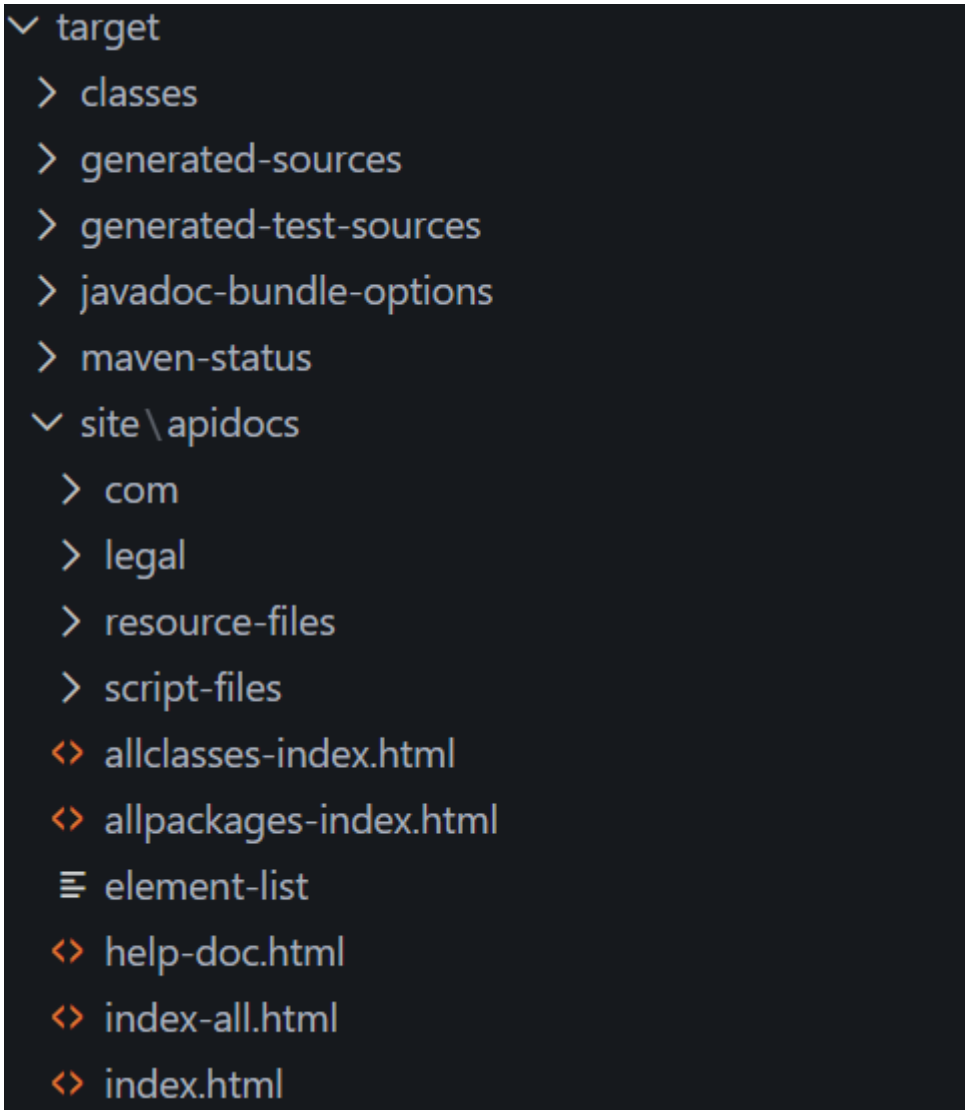
```
C:\Users\alumno\Documents\Real_Proyecto_Fenix\proyecto-fenix>mvn javadoc:javadoc
```

Hacemos un commit –m y luego un push para que se nos guarde.

```
C:\Users\alumno\Documents\Real_Proyecto_Fenix\proyecto-fenix>git commit -m "Añadido comentario y autoria"
```

Si el codigo esta perfecto y sin errores, despues de haber ejecutado el mvn javadoc:javadoc nos creara una carpeta llamada “target” y alli estara el index.html, que vendria a ser la “portada”

del proyecto por asi decirlo, y dentro de ella están, la lista de paquetes (`com.fenix`)
clases (como `ProcesadorUsuarios`),
métodos documentados y descripción que escribiste en comentarios.



Ahora usamos el mvn test, para ver si el programa compila.

```
C:\Users\alumno\Documents\Real_Proyecto_Fenix>mvn test
```

Si nos sale esto, quiere decir que el programa funciona correctamente.

```
------------------------
BUILD SUCCESS
------------------------
```

Recomendaciones (para evitar errores):

Limpia el proyecto (solución universal)

```
mvn clean
```

 Borra basura y errores raros

Ejecuta todo desde cero

```
mvn clean test
```

 Este es tu comando estrella

SI FALLA MAVEN

❌Error: “no POM in this directory”

 Solución:

```
cd proyecto-fenix/ cd permite cambiar de directorio en la terminal para ubicarse en la carpeta del proyecto antes de ejecutar comandos como Maven.”
```

❌Error de compilación Java

 Prueba:

```
mvn compile
```

✓ te dice exactamente dónde falla

❌Error con tests (JUnit)

 Prueba:

```
mvn test -e
```

✓ muestra error detallado

❌Maven muy raro / dependencias mal

 Fuerza actualización:

```
mvn clean install -U
```

COMANDOS ALTERNATIVOS ÚTILES

Ejecutar solo tests

```
mvn test
```

Compilar sin tests

```
mvn compile
```

□ Crear JAR

```
mvn package
```

□ Ejecutar TODO

```
mvn clean install
```

□ compila + test + empaqueta

□ Generar Javadoc

```
mvn javadoc:javadoc
```

□ SI FALLA GITHUB ACTIONS

❌Error: no encuentra el pom

□ Solución en `main.yml`:

```
working-directory: proyecto-fenix
```

❌Error de Java

□ Cambia versión:

```
java-version: '11'
```

❌Tests fallan

□ revisa tu clase de test
□ ejecuta local primero:

```
mvn test
```

□□ SI FALLA GIT

❌push rechazado

```
git pull origin main --rebase
git push
```

❌no hay cambios

□ significa:

```
nothing to commit
```

✓ no pasa nada, ya está todo subido

□ □ PROBLEMAS DETECTADOS

1□ Magic numbers should not be used

□ Uso de números mágicos

Ejemplo:

if (rol == 1) { ... }
❌Problema: no se sabe qué significa el 1
✓ Solución: usar constantes

private static final int ROL_ADMIN = 1;

2□ Rename this variable to match naming conventions

□ Variables mal nombradas

Ejemplo:

```
String n;
int r;
List dataList;
```

✕Problema: nombres poco claros

✓ Solución:

```
String nombre;
int rol;
List<String> usuarios;
```

3□ Methods should not be too long / Cognitive Complexity

□ Método demasiado largo o complejo

Ejemplo:

```
public String procesarTodo(...) {
  // mucho código junto
}
```

✕Problema:

- difícil de leer
- difícil de mantener

✓ Solución:

- dividir en métodos pequeños

```
procesarAdmin()
procesarInvitado()
```

RESPUESTAS:

- Uso de números mágicos ("Magic numbers should not be used"), lo que dificulta la comprensión del código.
- Nombres de variables poco descriptivos ("Rename this variable"), reduciendo la legibilidad.
- Métodos demasiado largos o complejos ("Methods should not be too long"), lo que dificulta el mantenimiento.